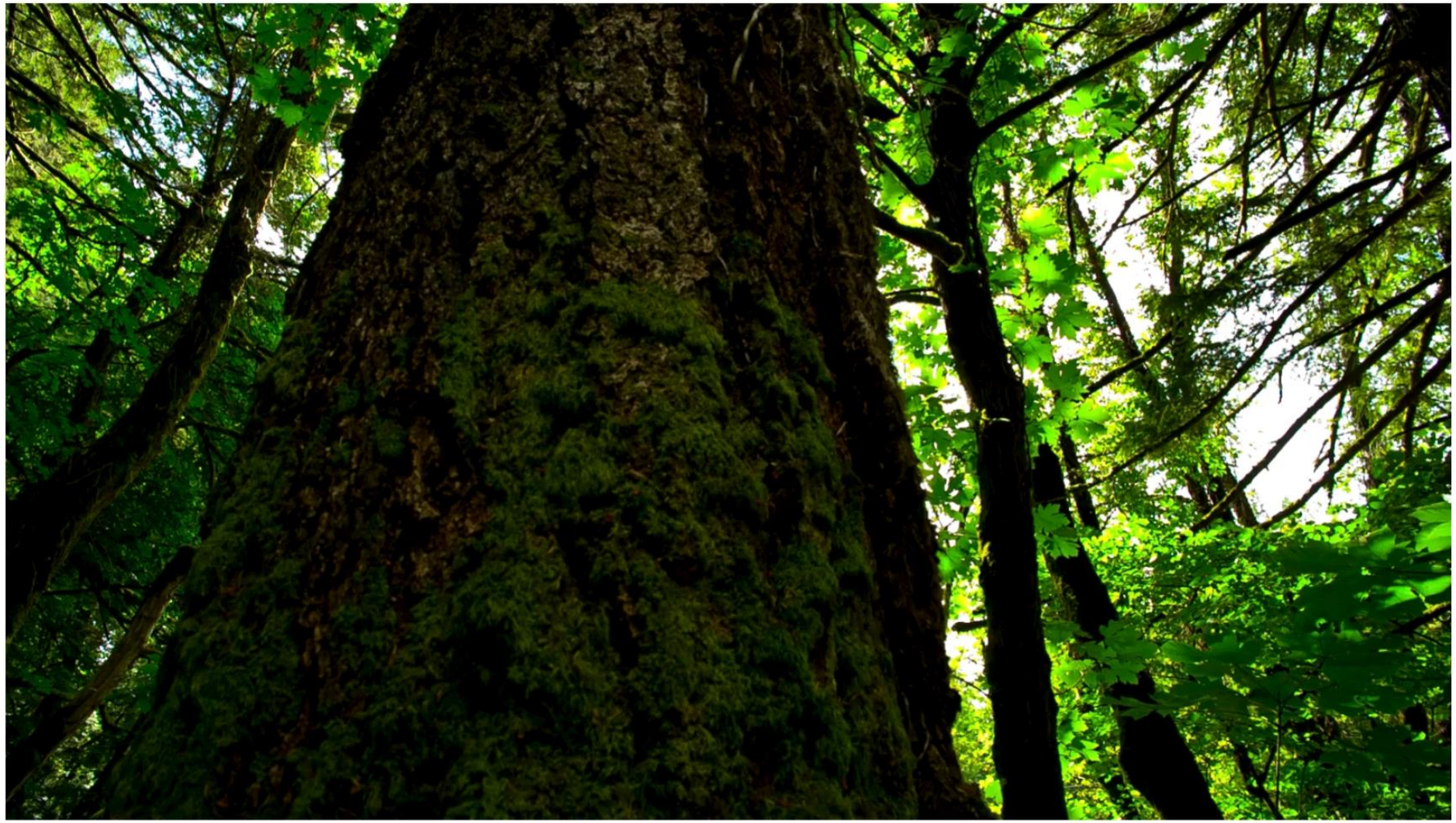
Random Forest for Classification



ENHANCE IMPACT

Introduction

- Hi Welcome to the course. We will be implementing one of the most popular machine learning algorithms: Random Forests. These supervised machine learning models are known for their power when it comes to both classification and regression tasks. In this lesson, we will focus on classification.
- So, how do random forests make decisions? What does the structure look like, and how do you prevent it from becoming too complex? Together, we'll explore these questions. I'll walk you through the steps of building a random forest classification model, using Python and Scikit-Learn.

By the end of this course, you'll understand:

- What a random forest is and how it works
- When to use random forest
- How to build and optimize random forest for classification
- How to evaluate and improve the performance

What is Random Forest?

- It builds an ensemble of decision trees, combining their predictions to produce more accurate and robust results.
- Based on bagging (Bootstrap Aggregating): Each tree is trained on a random subset of data (with replacement), reducing overfitting.
- Only a random subset of features is considered at each split, creating diverse trees.
- Predictions are combined via majority voting (classification) or averaging (regression).
- Handles both categorical and numerical data while reducing overfitting through diverse tree structures.

How Bootstrap Sampling Works

- Each tree in the Random Forest is trained on a bootstrap sample, which is created by random sampling with replacement.
- Since we sample with replacement, some original records may appear multiple times, while others may not appear at all.
- If the original dataset has 100 records, each bootstrap sample will also have 100 records, but about 63% of unique records will actually be used, and the rest will be duplicates.

Bootstrap aggregation (or bagging) in Random Forest Means:

1.Resampling with Replacement

- Each decision tree is trained on a random subset of the data.
- The subset is created by sampling with replacement, meaning some samples may appear multiple times, while others may not appear at all.
- Each subset is typically the same size as the original dataset.

2.Building Multiple Decision Trees

- Each resampled dataset is used to train a separate decision tree.
- These trees are fully grown (or pruned using max_depth) but trained on different data subsets.

3.Aggregating Predictions (Voting or Averaging)

- For classification, predictions are combined using majority voting (most common class wins).
- For regression, predictions are combined using the average of all trees' outputs.

✓***Reduces Overfitting*** *– Each tree is slightly different, so the ensemble is more robust.*
✓***Reduces Variance*** *– Small variations in data do not drastically change the model.*
✓***Improves Generalization*** *– Works well on unseen data.*

How Does Random Forest Work?

- **Splitting Criteria:**
  - At each split, the algorithm evaluates metrics like Gini Impurity or Information Gain to choose the best feature. The feature that best separates the data into distinct classes is selected.
- **Tree Growth:**
  - Each tree grows until a stopping criterion is met (e.g., max depth or minimum samples per leaf). This ensures trees don’t overfit the training data.
- **Feature Importance:**
  - Random Forests naturally rank features by their contribution to splitting decisions.

Confusion Matrix

- A **confusion matrix** is a table used to evaluate how well a classification model is performing. It compares the **actual** vs. **predicted** values.
- **Example Confusion Matrix**
- Let's say you're predicting whether an email is **Spam** or **Not Spam** (binary classification).

	Predicted: Not Spam	Predicted: Spam
Actual: Not Spam	True Negative (TN) ✅	False Positive (FP) ❌
Actual: Spam	False Negative (FN) ❌	True Positive (TP) ✅

Breaking Down Confusion Matrix Results

- **True Positive (TP)** ✅ → Model correctly predicts Spam as Spam.
- **True Negative (TN)** ✅ → Model correctly predicts Not Spam as Not Spam.
- **False Positive (FP)** ❌ **(Type I Error)** → Model wrongly predicts Not Spam as Spam (**"false alarm"**).
- **False Negative (FN)** ❌ **(Type II Error)** → Model wrongly predicts Spam as Not Spam (**"missed detection"**).

Key Metrics from Confusion Matrix

1. **Accuracy** → Overall correctness

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. **Precision (Positive Predictive Value)** → How often is a **positive prediction correct**?

$$Precision = \frac{TP}{TP + FP}$$

   High precision means fewer false positives!

3. **Recall (Sensitivity)** → How well does the model **catch positives**?

$$Recall = \frac{TP}{TP + FN}$$

   High recall means fewer false negatives!

4. **F1 Score** → Balance between **Precision & Recall**

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Real-World Example

- Imagine a **COVID-19 test**:

❌ **False Positive (FP)**: Healthy person wrongly told they have COVID.

❌ **False Negative (FN)**: Infected person wrongly told they are COVID-free (**dangerous!**).

- In **medical tests**, high **recall** is crucial (catching all real cases).
- In **email spam filters**, high **precision** is more important (not marking real emails as spam).

ROC and AUC Explained

- The **ROC (Receiver Operating Characteristic) curve** is a graph that shows how well a classification model separates two classes. It helps you **visualize the trade-off between sensitivity (recall) and specificity** at different thresholds.
- The **AUC (Area Under the Curve)** measures the overall performance of the model.

ROC and AUC (Continued)

Interpreting the ROC Curve

1. **A Perfect Model** → The curve reaches the top-left corner (**TPR = 1, FPR = 0**), meaning **100% accuracy**.
2. **A Random Model** → The curve is a **diagonal line** (50/50 guess).
3. **A Bad Model** → The curve dips **below the diagonal** (worse than random).

- **AUC (Area Under Curve)**
- **Higher AUC (closer to 1)** = Better model performance.
- **AUC = 0.5** means the model is **random**.
- **AUC < 0.5** means the model is **worse than random** (flipping predictions may improve it).

Purpose of train_test_split

- train_test_split is used to split a dataset into training and testing sets.
- The training set is used to train the model, while the test set is used to evaluate its performance on unseen data.
- This helps prevent overfitting and ensures that the model generalizes well to new inputs.

Purpose of Grid Search CV

- Grid Search CV is used for hyperparameter tuning, meaning it finds the best combination of model parameters to improve performance.
- It systematically tests different values using cross-validation, selecting the configuration that achieves the best results.
- Cross-validation is a technique used to test a machine learning model’s performance more reliably by splitting the dataset multiple times into training and validation sets.
- Instead of using a single train-test split, cross-validation rotates through different portions of the data, training on some parts and testing on others.

k-Fold Cross-Validation (Most Common)

How it Works:

- Split the data into k equal parts (folds).
- Train the model on k-1 folds and test it on the remaining fold.
- Repeat k times, using a different fold for testing each time.
- Average the results across all folds to get a final performance score.

Example: 5-Fold Cross-Validation

- Split data into 5 folds.
- Train on 4 folds, test on the remaining 1.
- Repeat this process 5 times (each fold gets a turn as the test set).
- Compute the average accuracy across all 5 runs.

Preventing Overfitting in Random Forest

Hyperparameter	Effect	Typical Fix
`n_estimators`	More trees reduce variance but increase training time.	Use a reasonable number (e.g., 100-500).
`max_depth`	Limits tree depth to prevent deep overfitting trees.	Use a depth of ~10-20 for large datasets.
`min_samples_split`	Minimum samples needed to split a node.	Increase to ensure meaningful splits.
`min_samples_leaf`	Minimum samples per leaf node.	Use higher values (e.g., 5-10) for smoother trees.
`max_features`	Limits the number of features used per split.	Use `sqrt` for classification, `log2` for regression.